

Phase -1.6 — Low-Level Design (LLD) & Module Breakdown

Objective

The High-Level Design identified the major components of the system.

The Low-Level Design (LLD) explains **how each component is internally organized**.

By the end of this phase, every directory, module, layer, and dependency should have a clearly defined responsibility before implementation begins.

What is Low-Level Design?

Low-Level Design describes the internal implementation structure of the application.

It answers questions such as:

- What modules exist?
- How do modules communicate?
- Which layer owns business logic?
- Where should database queries be written?
- Which components are reusable?
- What dependencies are allowed?

Unlike HLD, LLD focuses on developer-facing architecture.

Module Breakdown

The application is divided into independent business modules.

```
src/  
|  
├─ modules/  
|   ├─ auth/  
|   ├─ users/  
|   ├─ events/  
|   ├─ bookings/  
|   └─ payments/
```

```
|   └─ admin/
|
|─ common/
|─ config/
|─ database/
|─ cache/
|─ queues/
|─ docs/
└─ tests/
```

Each module should own everything related to its feature.

Internal Module Structure

Every business module follows the same layout.

```
events/
|
|─ controller.ts
|─ service.ts
|─ repository.ts
|─ routes.ts
|─ validator.ts
|─ dto.ts
|─ types.ts
|─ mapper.ts
└─ errors.ts
```

Consistency is more valuable than creativity.

Every module should look familiar.

Layer Responsibilities

Router Layer

Responsibilities:

- Define endpoints
- Apply middleware

- Forward requests

Must never contain business logic.

Controller Layer

Responsibilities:

- Receive validated requests
- Call services
- Return responses

Controllers should remain extremely thin.

Service Layer

Responsibilities:

- Business rules
- Validation beyond schema validation
- Transactions
- Workflow orchestration

This is the most important layer in the application.

Repository Layer

Responsibilities:

- Database interaction
- Queries
- Transactions
- Persistence

Repositories should know nothing about HTTP.

DTO Layer

DTOs define how data enters and leaves the application.

They isolate external API contracts from internal domain models.

Validation Layer

Responsible for:

- Request validation
- Response validation (optional)
- Input sanitization

Invalid data should never reach business logic.

Mapper Layer

Converts between:

- Database models
- DTOs
- Domain objects

This prevents leaking database structures directly through the API.

Error Layer

Each module may define domain-specific errors.

Examples:

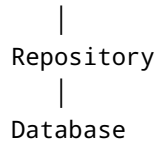
- EventNotFoundError
- SeatAlreadyBookedError
- BookingExpiredError

These are converted into HTTP responses by the global error handler.

Dependency Rules

Dependencies should only move downward.

```
Router
|
Controller
|
Service
```



A repository must never call a controller.

A service must never depend on Express.

Business logic should remain independent of framework-specific code whenever possible.

Shared Components

The `common` directory contains reusable infrastructure shared across all modules.

Examples include:

- Middleware
- Error classes
- Logger
- Utility functions
- Base types
- Response helpers
- Constants

Feature-specific code should never be placed here.

Configuration Layer

The `config` module is responsible for:

- Environment variables
- Application configuration
- Feature flags
- Runtime settings

No other part of the application should access `process.env` directly.

Database Layer

Responsible for:

- Prisma Client
- Migrations
- Seed scripts
- Database initialization

Business modules interact with the database only through repositories.

Queue Layer

Responsible for:

- Queue definitions
- Workers
- Job processors
- Retry policies

Business modules enqueue work rather than performing long-running tasks synchronously.

Cache Layer

Responsible for:

- Redis client
- Cache helpers
- Cache keys
- Cache invalidation

Caching logic should remain centralized.

Documentation Layer

Contains:

- OpenAPI specification
- Architecture diagrams
- ADRs (Architecture Decision Records)
- API examples

Documentation is treated as part of the codebase.

Testing Layer

Testing is organized by scope rather than by technology.

```
tests/  
|  
├─ unit/  
├─ integration/  
├─ e2e/  
└─ fixtures/
```

This keeps the testing strategy clear and scalable.

Architectural Rules

Every contributor should follow these rules:

1. Controllers do not contain business logic.
 2. Services never return HTTP responses.
 3. Repositories never contain business rules.
 4. Validation occurs before business logic.
 5. Errors are handled centrally.
 6. Modules communicate through services, not direct database access.
 7. Shared code belongs in `common`, not duplicated across modules.
 8. External dependencies should be isolated behind abstractions where practical.
-

Deliverables

At the end of this phase, the internal structure of the application is fully defined.

Every future implementation task will map directly to one of these modules and layers.

The next phase, **Phase -1.7 — API Design & REST Contract**, will define every endpoint, request, response, status code, and error contract before implementation begins.